

```

import copy
btCalls = 0 # total times BACKTRACK was called
btFailures = 0 # total times BACKTRACK returned failure

def buildDomains(board):
    domains = {}
    for r in range(9):
        for c in range(9):
            if board[r][c] != 0:
                domains[(r, c)] = [board[r][c]]
            else:
                domains[(r, c)] = list(range(1, 10))
    # number from 1 to 9
    return domains

def getNeighbours(r, c):
    neighbours = set()
    for col in range(9):
        # every cell in the same row
        if col != c:
            neighbours.add((r, col))
    for row in range(9):
        # every cell in the same column
        if row != r:
            neighbours.add((row, c))
    boxR = 3 * (r // 3)
    boxC = 3 * (c // 3)
    for row in range(boxR, boxR + 3):
        for col in range(boxC, boxC + 3):
            if (row, col) != (r, c):
                neighbours.add((row, col))
    return neighbours

def makeConsistant(domains, xi, xj):
    pruned = False
    for val in list(domains[xi]):
        if all(val == w for w in domains[xj]):
            domains[xi].remove(val)
    pruned = True
    return pruned

def AC3(domains):
    queue = []
    for cell in domains:
        for neighbours in getNeighbours(cell[0], cell[1]):
            queue.append((cell, neighbours))
    while queue:
        xi, xj = queue.pop(0)
        if makeConsistant(domains, xi, xj):
            if len(domains[xi]) == 0:
                return False # xi has no values left
            for neighbours in getNeighbours(xi[0], xi[1]):
                if neighbours != xj:
                    queue.append((neighbours, xi))
    return True # all arcs are now arc consistent

def forwardCheck(domains, cell, val):
    newDomains = copy.deepcopy(domains)
    newDomains[cell] = [val]
    for neighbours in getNeighbours(cell[0], cell[1]):
        if val in newDomains[neighbours]:
            newDomains[neighbours].remove(val)
        if len(newDomains[neighbours]) == 0:
            return None
    return newDomains

# MRV
def selectCell(domains, assignment):
    unassigned = [cell for cell in domains if cell not in assignment]
    return min(unassigned, key=lambda cell: len(domains[cell]))

def backtrack(assignment, domains):
    global btCalls, btFailures
    btCalls += 1
    if len(assignment) == 81:
        return assignment
    cell = selectCell(domains, assignment)
    for val in domains[cell]:
        # check every remaining value
        assignment[cell] = val
        newDomains = forwardCheck(domains, cell, val)
        if newDomains is not None:
            result = backtrack(assignment, newDomains)
            if result is not None:
                return result
    del assignment[cell]
    btFailures += 1
    return None

def solveSudoku(board):
    global btCalls, btFailures
    btCalls = 0 # reset counters
    btFailures = 0
    domains = buildDomains(board)
    if not AC3(domains):
        print("AC-3: Puzzle has no solution")
        return None
    assignment = {}
    for cell, dom in domains.items():
        if len(dom) == 1:
            assignment[cell] = dom[0]
    result = backtrack(assignment, domains)
    if result is None:
        return None
    solvedBoard = [[0] * 9 for _ in range(9)]
    for (r, c), val in result.items():
        solvedBoard[r][c] = val
    return solvedBoard

def printBoard(board, title=""):
    if title:
        print("\n <----- " + title + " ----->\n")
    for r in range(9):
        if r % 3 == 0 and r != 0:
            print("  _____")
        row = ""
        for c in range(9):
            if c % 3 == 0 and c != 0:
                row += " | "
            val = board[r][c]
            row += (str(val) if val != 0 else ".") + " "
        print(row)

def readBoardFromFile(filename):
    board = []
    try:
        with open(filename, "r") as f:
            for line in f:
                line = line.strip()
                if len(line) == 9:
                    board.append([int(ch) for ch in line])
    except FileNotFoundError:
        print("The file " + filename + " was not found")
    return board

def runSolver(name, filename):
    board = readBoardFromFile(filename)
    if not board:
        return
    print("\n<--- BOARD : " + name + " ----->")
    printBoard(board, "Puzzle")
    solved = solveSudoku(board)
    if solved:
        printBoard(solved, "Solution")
        print("\n<----- Stats ----->\n")
        print("BACKTRACK calls : " + str(btCalls))
        print("BACKTRACK failures: " + str(btFailures))
        if btCalls <= 2 and btFailures == 0:
            print("AC-3 solved the most with minimal search")
        elif btFailures == 0:
            print("Forward checking saw every dead-end early")
        else:
            print(str(btFailures) + " dead-ends required backtracking")
    else:
        print("No solution found!")
    if __name__ == "__main__":
        runSolver("Easy", "easy.txt")
        runSolver("Medium", "medium.txt")
        runSolver("Hard", "hard.txt")
        runSolver("Very Hard", "veryhard.txt")

```